

Session 2 — Histoire du logiciel libre (1950-1990)

Objectifs de la séance

- Retracer l'évolution de l'industrie du logiciel des années 1950 aux années 1990.
- Expliquer le contexte d'émergence du mouvement du logiciel libre.
- Comprendre la philosophie Unix et son influence durable.
- Identifier les figures fondatrices et leurs contributions.
- Situer les arguments du *GNU Manifesto*.

Partie 1 — La préhistoire du logiciel (1950-1970)

L'ère des mainframes

Dans les années 1950-60, les ordinateurs sont des **machines géantes et coûteuses** (*mainframes*) que seuls gouvernements, grandes entreprises et universités peuvent se payer. Le logiciel est vu comme un **accessoire du matériel** : on achète une machine, le constructeur fournit le système avec.

Les acteurs dominants :

- **IBM**, quasi-monopolistique sur les mainframes.
- **DEC**, qui lance le marché des *miniordinateurs* (PDP-1, PDP-7, PDP-11) à partir des années 1960.
- Les grands centres de recherche : **MIT**, **Berkeley**, **Bell Labs** (AT&T).

Anecdote souvent citée : « *I think there is a world market for maybe five computers.* » — attribuée (à tort) à Thomas Watson, IBM, 1943.

Le logiciel, un bien naturellement partagé

Avant la fin des années 1960, **partager le code est la norme**, et pas le contraire :

- Le logiciel est livré avec le matériel, sans licence restrictive.
- IBM distribue le code source de ses OS.
- Les utilisateurs échangent leurs programmes dans des groupes d'utilisateurs (**SHARE** pour IBM, **DECUS** pour DEC).
- On compare le code à une « recette de cuisine » — on l'améliore, on la passe au voisin.

1969 : le tournant IBM (*unbundling*)

En 1969, IBM annonce son **unbundling** : la séparation commerciale du matériel et du logiciel. Désormais, le logiciel est **facturé séparément**.

Les raisons :

- **Pression antitrust** du gouvernement américain sur IBM.
- **Opportunité commerciale** : la demande existe, IBM la monétise.

Conséquence historique : le logiciel passe du statut de *bien commun* partagé dans un écosystème technique à celui de **marchandise protégée**. C'est le début de l'industrie du logiciel propriétaire telle qu'on la connaît encore.

Partie 2 — Unix : naissance d'un écosystème

Les origines (1969)

Unix naît **aux Bell Labs d'AT&T**, créé par **Ken Thompson** et **Dennis Ritchie**. Contexte :

- 1965 : MIT, Bell Labs et GE lancent **MULTICS**, système ambitieux.
- 1969 : Bell Labs se retire de MULTICS (trop complexe, trop lent).

- Thompson récupère un **PDP-7** inutilisé et écrit un système minimaliste (au départ pour faire tourner son jeu *Space Travel*). Brian Kernighan baptise le tout **UNICS** — jeu de mots avec MULTICS (Uniplexed vs Multiplexed *Information and Computing Service*).
- 1971-1973 : Ritchie conçoit le langage **C** (issu du langage B).
- 1973 : Unix est **réécrit en C**, ce qui le rend **portable** — une innovation majeure à l'époque, où chaque OS était lié à son matériel.

Pourquoi AT&T laisse Unix circuler librement ? AT&T est sous **Consent Decree** (1956) : pour garder son monopole téléphonique, la firme s'est engagée à ne pas commercialiser d'autres produits. Unix est donc distribué aux universités pour quelques centaines de dollars (« prix du média et des frais d'envoi »), avec le code source.

Caractéristiques techniques

Unix apporte plusieurs innovations qui deviendront des standards :

- Noyau simple, modulaire, écrit dans un langage de haut niveau (C).
- **Portable** d'une machine à l'autre.
- **Multi-utilisateur, multi-tâches**.
- Abstraction fondamentale : « *Everything is a file* » — fichiers, périphériques, sockets, processus sont manipulés via la même API.
- Processus, mémoire virtuelle, pipes entre programmes.

Architecture

Trois couches concentriques :

- **Kernel** : gestion des processus, des périphériques, du système de fichiers.
- **Shell** : interface utilisateur en ligne de commande, langage de script.
- **Outils** : l'immense ensemble des utilitaires (ls, grep, awk, sed, make...).

Diffusion académique

Comme AT&T ne peut pas le commercialiser, **Unix se répand dans les universités** avec son code source, et devient le **standard académique** dans les années 1970. Cela va avoir deux conséquences majeures : (1) une génération entière d'ingénieurs est formée sur Unix, (2) un fork important naît à **Berkeley** en 1977 : **BSD**.

Partie 3 — La philosophie Unix

Les pipes (1973)

L'invention des **pipes** par **Doug McIlroy** est le geste fondateur de la philosophie Unix. Une pipe (|) envoie la sortie d'un programme à l'entrée d'un autre. Exemple : `cat fichier.log | grep ERROR | sort | uniq -c`.

« Was the notion of toolbox there before pipes ? — No. Or did pipes create it ? — **Pipes created it.** »

Les pipes rendent naturel le fait d'écrire de **petits programmes spécialisés**, puis de les **composer** pour résoudre un problème plus large.

Les règles de McIlroy (1978)

1. **Make each program do one thing well.** Pour un nouveau besoin, écrire un nouvel outil plutôt que complexifier un outil existant.
2. **Expect the output of every program to become the input to another.** Sorties structurées, pas de bavardage.
3. **Design and build software to be tried early** — prototyper vite, jeter ce qui ne marche pas.

4. **Use tools** plutôt que de la main-d'œuvre peu qualifiée pour alléger une tâche.

Les règles d'Eric S. Raymond (*The Art of Unix Programming*, 2003)

Énoncés qui formalisent la culture Unix :

- **Modularity** : écrire des pièces simples reliées par des interfaces propres.
- **Clarity** : la clarté prime sur la virtuosité.
- **Composition** : concevoir des programmes qui se connectent à d'autres programmes.
- **Simplicity** : commencer simple, ajouter de la complexité seulement quand c'est indispensable.
- **Transparency** : faciliter l'inspection du comportement.
- **Robustness** : fille naturelle de la transparence et de la simplicité.
- **Least Surprise** : faire ce que l'utilisateur attend.
- **Silence** : si un programme n'a rien d'intéressant à dire, il doit se taire.

« Worse is Better » (Richard Gabriel, 1989)

Dans *The Rise of Worse is Better*, **Richard Gabriel** oppose deux manières de concevoir un logiciel — qu'il nomme d'après les lieux qui les incarnaient à l'époque :

	MIT / Stanford (« The Right Thing »)	New Jersey (Unix, Bell Labs)
Priorités (ordre)	Justesse > cohérence > simplicité d'interface > simplicité d'implémentation	Simplicité d'implémentation > simplicité d'interface > cohérence > justesse
Face à un cas pénible	On complique l'implémentation pour couvrir le cas proprement	On simplifie l'interface, quitte à laisser le cas de côté
Exemple canonique	Lisp Machines, systèmes du MIT AI Lab	Unix, C
Pari implicite	La qualité finira par l'emporter	« <i>Good enough</i> » se diffuse, et s'améliorera ensuite

Pour Gabriel, la thèse contre-intuitive est que **la seconde approche gagne dans la durée**. Un outil « imparfait mais simple » se **porte** plus vite sur de nouvelles machines, se **comprend** plus vite par de nouveaux développeurs, et se **diffuse** plus largement. Cette masse d'utilisateurs et de contributeurs finit par **combler les défauts initiaux** — alors que le concurrent « parfait » reste confidentiel et stagne.

■ « Unix and C are the ultimate computer viruses. » — Richard Gabriel

Ce que le débat recouvre vraiment

Une lecture classique réduit l'opposition à *simplicité contre correction*. **Yossi Kreinin** (*What « Worse is Better » is really about*) montre qu'il y a en fait un **désaccord plus profond, sur le fonctionnement du marché** et des systèmes socio-techniques :

- Les tenants de *The Right Thing* partent du principe que **le marché est défaillant** : il récompense l'inférieur, il faut donc viser la qualité d'emblée, quitte à rester minoritaire.
- Les tenants de *Worse is Better* traitent le marché comme un **mécanisme évolutionnaire** : **survivre** et **se reproduire** (portabilité, compatibilité, adoption) comptent plus que la pureté du design — parce que ce sont précisément les logiciels qui survivent qui sont ensuite améliorés par leurs utilisateurs.

C'est pour cela que l'opposition refait surface régulièrement dans l'histoire de l'informatique : **x86 vs RISC** (la compatibilité descendante et les volumes l'emportent sur l'élégance), **Windows vs Mac**

pendant longtemps, **JavaScript** face à des langages plus propres... et surtout, pour ce cours, **Linux vs GNU Hurd** — qu'on va voir en session 3.

Pourquoi c'est essentiel pour comprendre l'open source. Le mouvement du logiciel libre a historiquement pris les deux postures :

- **GNU Hurd** (FSF, 1990) est un projet *Right Thing* : architecture à micro-noyau élégante, ambitieuse, jamais vraiment terminée.
- **Linux** (1991) est un projet *Worse is Better* : noyau monolithique « à l'ancienne », compatible Unix, publié tôt, amélioré par des milliers de contributeurs. **Il a gagné** — pas parce qu'il était techniquement supérieur au départ, mais parce qu'il était suffisamment bon pour être adopté, et qu'il a ensuite accumulé les contributions.

C'est exactement la même logique qu'Eric Raymond formalisera en 1997 sous le nom de « **Cathédrale vs Bazar** » (session 3).

Berkeley et BSD

En 1976-77, Ken Thompson effectue un sabbatique à UC Berkeley. **Bill Joy** et **Chuck Haley** démarrent la **Berkeley Software Distribution (BSD)** ; le **CSRG** (*Computer Systems Research Group*) est formellement établi en **1980**, financé par la DARPA.

Contributions majeures de BSD (et pérennes) :

- **Pile TCP/IP** (BSD 4.2, 1983) — base du réseau internet mondial.
- Outils : **vi**, **csch**, **sendmail**.
- **Mémoire virtuelle**, **sockets** réseau.
- Base technique de **SunOS/Solaris**, **NeXTSTEP**, et finalement **macOS** et **iOS**.

BSD adopte une **licence permissive** (« faites ce que vous voulez, créditez-nous ») — un modèle philosophiquement distinct de la GPL (voir session 4).

La culture hacker du MIT

En parallèle, au **MIT AI Lab**, une communauté de programmeurs développe une culture singulière :

- **Partage du code** comme norme absolue.
- **Méritocratie technique** : ce qui compte, c'est la qualité du hack.
- « **L'information veut être libre.** »
- Le système **ITS** (*Incompatible Timesharing System*) n'a même pas de mots de passe : tout est ouvert à tous.

Richard Stallman y entre comme programmeur en 1971. Cette culture constitue son univers de référence ; sa désintégration au début des années 1980 sera le point de bascule.

Partie 4 — La fermeture du logiciel (1970-1983)

L'essor du logiciel commercial

Quelques jalons marquants :

Année	Événement
1975	Fondation de Microsoft (Gates, Allen)
1976	« Open Letter to Hobbyists » de Bill Gates
1976	Fondation d' Apple
1979	VisiCalc , premier <i>killer app</i> sur micro-ordinateur

Année	Événement
1980	Amendement du Copyright Act américain : le logiciel est explicitement protégé
1981	IBM PC + MS-DOS
1982	Éclatement d'AT&T : Unix peut désormais être commercialisé

La lettre ouverte de Bill Gates (1976)

Gates s'adresse aux hobbyists (club MITS Altair) qui s'échangent des copies de l'interpréteur BASIC de Microsoft :

« *As the majority of hobbyists must be aware, most of you steal your software. [...] Who can afford to do professional work for nothing ?* »

Ses arguments : le développement logiciel coûte cher, le « piratage » empêche l'innovation, les développeurs méritent d'être payés. Cette lettre marque **le changement de statut symbolique** du logiciel : il devient explicitement une **marchandise**.

L'effondrement du MIT AI Lab

Au début des années 1980, le laboratoire d'IA du MIT se vide :

- Deux **spin-offs** commerciaux absorbent les meilleurs hackers : **Symbolics** et **LMI**.
- Le code qu'ils écrivent devient **propriétaire**.
- La communauté se disloque ; Stallman reste quasiment seul.

Le mythe fondateur : l'imprimante Xerox (vers 1980)

Stallman voulait modifier le pilote d'une **imprimante Xerox 9700** pour qu'elle prévienne quand le papier bourrait. Xerox refuse de lui communiquer le code source : **il n'a plus le droit de réparer l'outil qu'il utilise**.

« *J'ai réalisé que le logiciel propriétaire était une injustice sociale.* »

L'épisode, réel ou largement reconstruit, deviendra le **récit fondateur** du mouvement du logiciel libre : un programmeur empêché d'entraider son propre voisin parce qu'un contrat de licence l'interdit.

Partie 5 — Richard Stallman et la naissance du libre

L'homme

Richard Matthew Stallman (RMS) — né en 1953 à New York, au MIT AI Lab à partir de 1971, brillant développeur (auteur d'**Emacs**), dernier tenant de la culture hacker du laboratoire après son effondrement.

Sa vision se forme à partir de l'expérience Xerox et de la dislocation du MIT :

- Le logiciel propriétaire est une **injustice sociale**, car il empêche l'entraide.
- Les utilisateurs méritent la **liberté**.
- Il faut **reconstruire** un écosystème libre complet, utilisable en production.
- Le sujet est **éthique**, pas simplement technique ou économique.

27 septembre 1983 : l'annonce de GNU

Stallman publie sur Usenet :

« *Starting this Thanksgiving I am going to write a complete Unix-compatible software system called GNU (for Gnu's Not Unix), and give it away free to everyone who can use it.* »

Deux décisions stratégiques clés :

1. **Créer un OS complet** (pas juste un outil isolé).
2. **Être compatible Unix** pour faciliter l'adoption : les utilisateurs existants peuvent migrer sans tout réapprendre.

Le projet GNU

Composants produits, dont plusieurs sont aujourd'hui encore standard :

- **GNU Emacs** (1984) — éditeur extensible.
- **GCC** (1987) — compilateur C (puis C++, Ada, Fortran, etc.).
- **GDB, GNU Make, Bash, Coreutils** (ls, cat, grep, cp...).

Ce qui manque encore à la fin des années 80 : **le noyau**. Le projet **GNU Hurd** est lancé en 1990 mais, pour des raisons de choix techniques ambitieux (architecture à micro-noyau), ne sera jamais véritablement fini. Ce trou sera comblé — fortuitement — par **Linux** (session 3).

Le GNU Manifesto (1985)

Publié dans *Dr. Dobbs's Journal*, le manifeste expose les fondements politiques du projet. Quatre arguments à retenir :

1. Les utilisateurs seront **libres** de partager et modifier leurs outils.
2. Le logiciel **sera meilleur** grâce à la collaboration (intuition qui sera confirmée par *The Cathedral and the Bazaar* en 1997 — session 3).
3. Les développeurs **pourront gagner leur vie** autrement (services, support, formation, adaptation).
4. C'est une question d'**éthique** : le logiciel propriétaire divise les utilisateurs et les prive de leur autonomie.

« I consider that the Golden Rule requires that if I like a program I must share it with other people who like it. »

La Free Software Foundation (1985)

Stallman fonde la **FSF** pour donner une structure durable au mouvement :

- Promotion politique et philosophique du logiciel libre.
- Financement du développement de GNU (emploi de développeurs).
- **Défense juridique** des licences libres (notamment GPL).
- Définition de référence : les **4 libertés** (vues en session 1).

1989 : la GPL, un « hack juridique »

La **GNU General Public License** v1 est publiée en 1989 (v2 en 1991, v3 en 2007).

Le mécanisme du copyleft (fondement juridique détaillé en session 4) :

1. Le code est couvert par le **droit d'auteur** (copyright) : Stallman, comme tout auteur, en dispose.
2. La licence **utilise** ce droit pour **garantir** les 4 libertés aux utilisateurs.
3. Toute version modifiée redistribuée doit être publiée **sous GPL**.
4. Les libertés sont donc **virales** et **irrévocables** : elles se propagent avec le code.

■ **Le copyleft retourne le copyright contre lui-même** pour garantir la liberté.

X Window System (1985) — un autre modèle

Le **X Consortium** au MIT publie **X11** sous une licence **très permissive** (ancêtre de la MIT License). Pas de copyleft, pas de viralité, réutilisation quasi libre même dans du propriétaire. C'est l'autre grande tradition du libre, qu'on opposera au copyleft en session 4.

Fin des années 80 : l'écosystème s'étoffe

Année	Événement
1987	GCC 1.0 — compilateur C libre de qualité industrielle
1987	Perl 1.0 (Larry Wall)
1988	X Window System devient libre (MIT License)
1989	GPL v1
1989	Cygnus Solutions — première entreprise commerciale bâtie sur GNU
1990	Début du projet GNU Hurd

Situation en 1990 : tous les composants d'un système libre utilisable existent... **sauf le noyau**. La scène est prête pour le coup de théâtre de 1991 (session 3).

Points de vigilance

- **Attention à l'anachronisme** : dans les années 1960, « partager le code » n'a rien d'un choix militant — c'est simplement la norme technique. Le militantisme apparaît par réaction, après l'*unbundling* et la fermeture progressive.
- **Unix ≠ logiciel libre** : Unix a été distribué largement, mais sous un régime juridique ambigu (licences AT&T). Il n'était pas libre au sens de la FSF. BSD le sera vraiment plus tard (après procès UNIX System Labs vs Berkeley, 1992-94).
- **GNU ≠ Linux** : GNU est l'ensemble des outils utilisateur (compilateur, shell, utilitaires) ; Linux est le noyau. C'est pour cela que Stallman parle de « **GNU/Linux** ».
- « **Free** » reste piégeux en anglais (session 1) : la FSF se bat constamment contre le contresens « gratuit ».

Ce qu'il faut retenir

Année	Événement	Signification
< 1969	Partage du code = norme implicite	Pas d'idéologie : c'est juste la pratique technique de l'époque
1969	<i>Unbundling</i> IBM ; naissance d'Unix (Bell Labs)	Double tournant : le logiciel devient marchandise, un OS portable apparaît
1973	Unix réécrit en C ; invention des pipes	Fondation de la philosophie Unix (modularité, composition)
1976	« Open Letter to Hobbyists » (Gates)	Le logiciel est explicitement affirmé comme marchandise
1977	BSD à Berkeley	Seconde tradition : permissive, universitaire
1980	Copyright Act (US) inclut le logiciel	Cadre juridique de la fermeture
1983	Stallman annonce le projet GNU	Réaction militante à la fermeture
1985	FSF créée ; GNU Manifesto	Structure et doctrine du mouvement
1987	GCC 1.0	L'écosystème GNU devient crédible
1989	GPL v1	Invention juridique du copyleft
1990	Écosystème GNU quasi complet, il manque le noyau	La scène est prête pour 1991
1991	(suite en session 3)	

Pour aller plus loin

- GNU Manifesto : <https://www.gnu.org/gnu/manifesto.html>
- Peter Salus, *A Quarter Century of Unix* (1994).
- Eric S. Raymond, *The Art of Unix Programming* (2003) — <http://www.catb.org/~esr/writings/taoup/>
- Eric S. Raymond, *The Cathedral and the Bazaar* — <http://www.catb.org/~esr/writings/cathedral-bazaar/>
- Rob Pike, *Unix History* (conférence, 2018) : https://www.youtube.com/watch?v=_2NI6t2r_Hs
- Brian Kernighan sur Unix (LWN) : <https://lwn.net/Articles/881431/>